

메타코딩의 자율주행 개발 일기

"AI를 처음 만나는 사람도, 끝까지 읽으면 이해하게 되는 이야기"

5주간의 실패와 성장을 따라가다 보면, 어느새 강화학습의 핵심이 손에 잡힙니다.

목차

- 프롤로그: 갑작스러운 프로젝트
- 0화: AI 없이 해봤더니... (v0)
- 1화: 첫 번째 AI, 첫 번째 함정 (v1)
- 2화: 실패에서 배우는 법 (v2)
- 3화: 진짜 실력을 증명하라 (v3)
- 4화: 더 빠르게, 더 똑똑하게 (v4)
- 5화: 현장에서 살아남기 (v5)
- 6화: 73%에서 90%로 — 남은 여정
- 에필로그: 메타코딩이 진짜 배운 것

프롤로그: 갑작스러운 프로젝트

 17 어느 월요일 아침

사무실에 들어서자마자 팀장님이 손짓했습니다.

"메타코딩 씨, 잠깐 와봐요."

회의실 문을 열었더니, 테이블 위에 작은 로봇 청소기가 놓여 있었습니다. 바퀴가 달린 작은 플라스틱 덩어리. 웬지 그게 오늘부터 메타코딩의 운명을 바꿔놓을 것 같은 불길한 예감이 들었습니다.

"이거 보이죠? 우리 회사에서 자율주행 청소 로봇을 만들기로 했어요."

팀장님이 로봇 청소기를 톡톡 두드렸습니다.

"메타코딩 씨가 AI 파트를 담당해주세요."

순간 메타코딩의 머릿속이 하얘졌습니다.

'AI? 나를요?'

강화학습은 대학교 때 교수님이 PPT로 설명해주신 적이 있었습니다. 솔직히 절반도 이해를 못 했습니다. '벨만 방정식'이라는 글자가 화면에 떴을 때 바로 졸음이 밀려왔던 기억만 생생합니다. 그걸로 실제 제품을 만든다고요?

"걱정 마세요. 처음부터 실제 로봇에 넣을 필요 없어요. 일단 컴퓨터 시뮬레이터로 먼저 만들어봐요. 2주 안에 간단한 데모만 보여주시면 됩니다!"

팀장님은 밝게 웃으며 회의실을 나갔습니다. 메타코딩은 로봇 청소기와 둘이 남았습니다. 로봇 청소기의 동그란 버튼이 마치 눈처럼 보였습니다. '너도 나처럼 길을 모르는구나.'

🏠 집에 돌아와서

그날 밤, 메타코딩은 노트북 앞에 앉아 인터넷을 뒤지기 시작했습니다.

"자율주행 강화학습 입문"

검색 결과에 **DQN**이라는 단어가 계속 나왔습니다. Deep Q-Network. 풀어보면 "깊은 Q 네트워크"인데, 이름만으로는 도무지 감이 잡히지 않았습니다.

그런데 읽다 보니 흥미로운 점이 있었습니다. 2013년에 DeepMind라는 회사가 이 DQN으로 아타리 게임을 사람보다 잘 하는 AI를 만들었다는 것입니다. 게임 AI랑 청소 로봇이 비슷하지 않을까? 둘 다 어떤 공간에서 목표를 향해 이동하는 거잖아.

"좋아, 일단 뭔가 만들어보자. 모르겠으면 가장 단순한 것부터!"

메타코딩은 노트에 첫 번째 줄을 적었습니다.

"Step 1: AI 없이 규칙만으로 만들어보자."

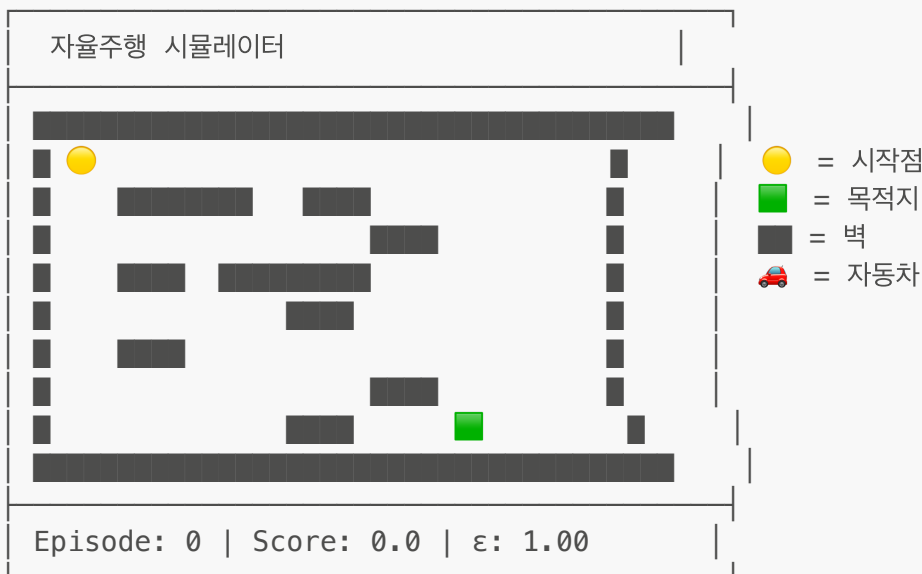
🎯 시뮬레이터 설계

다음날 출근해서 메타코딩은 시뮬레이터부터 설계했습니다. 진짜 로봇을 만들기 전에, 컴퓨터 화면에서 먼저 실험해보는 것이죠.

목표는 단순합니다:

- 시작점 ● (노란색)에서 출발
- 목적지 ■ (초록색)까지 도달
- 벽 ■ (빨간색)에 부딪히면 안 됨

[시뮬레이터 화면 구조]



30×30 칸의 격자 세상. 이 작은 세상에서 자동차가 벽을 피해 목적지를 찾아야 합니다.

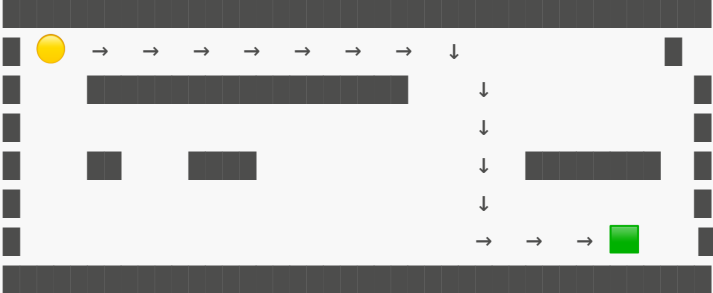
그런데 자동차를 어떻게 움직이게 할지가 문제였습니다. 메타코딩은 두 가지 방법을 생각했습니다.

	방법 1: 규칙	방법 2: AI
원리	"목적지가 오른쪽이면 오른쪽으로!"	"좋은 행동엔 칭찬, 나쁜 행동엔 꾸중"
장점	간단하고 빠르게 만들 수 있음	복잡한 상황도 스스로 해결
단점	예외 상황에 약함	만들기 어렵고 학습이 필요

"일단 방법 1부터 해보자. 그게 제일 쉬우니까."

0화: AI 없이 해봤더니... (v0)

0화에서 만드는 것: simulator-v0



전략: "목적지 방향으로 if 문 이동!"
단점: 복잡한 미로에서는 갇힘

💡 가장 단순한 아이디어

메타코딩은 종이에 규칙을 적었습니다.

규칙 1: 목적지가 오른쪽에 있으면 → 오른쪽으로 **규칙 2:** 목적지가 아래쪽에 있으면 → 아래쪽으로 **규칙 3:** 벽에 막혔으면 → 다른 방향 시도

마치 나침반을 손에 들고 항상 목적지 방향으로 걸어가되, 벽이 나오면 돌아가는 것이죠. 너무 간단해 보였지만, 메타코딩은 일단 코드를 짰습니다.

🔧 v0 코드 소개

```
def decide_action(self, environment):
    goal_x, goal_y = environment.goal_pos

    # 목적지까지의 거리 계산
    dx = goal_x - self.x    # 양수 = 오른쪽, 음수 = 왼쪽
    dy = goal_y - self.y    # 양수 = 아래쪽, 음수 = 위쪽

    # 우선순위 방향 결정
```

```

preferred = []
if dx > 0: preferred.append(RIGHT)
elif dx < 0: preferred.append(LEFT)
if dy > 0: preferred.append(DOWN)
elif dy < 0: preferred.append(UP)

# 우선순위로 벽이 없는 방향으로 이동
for direction in preferred:
    next_x, next_y = 다음_위치(direction)
    if not environment.is_wall(next_x, next_y):
        return direction # 이 방향으로!

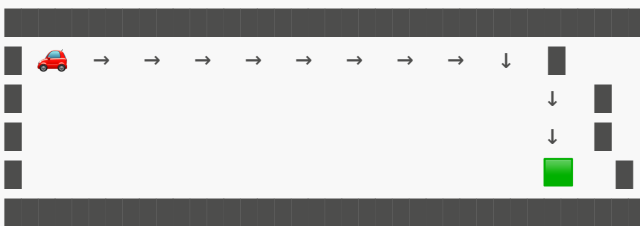
```

코드는 짧고 명쾌했습니다. "목적지가 어디 있나 보고, 그 방향으로 가되, 벽이면 다른 데로 가라." 사람이 보기에 자연스러운 논리죠.

🎮 v0 실행해보니...

단순한 맵에서는 놀랍도록 잘 작동했습니다.

[단순한 맵 - v0 성공!]



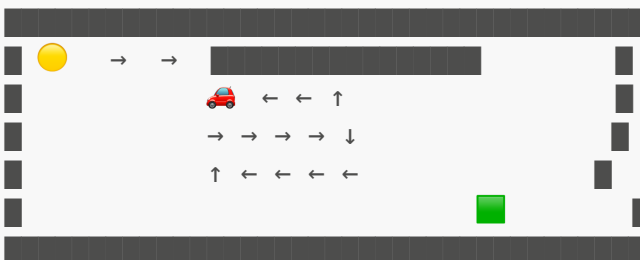
결과: ✅ 성공! (87걸음)

"오! 되는데?"

메타코딩은 신이 났습니다. 화면 속 파란 자동차가 목적지까지 매끄럽게 달려갔으니까요.

하지만 복잡한 미로 맵에서 실행했더니...

[복잡한 미로 - v0 실패!]



← 자동차가
같은 자리를
뱅뱅 돌고 있음!

결과: ❌ 실패! 시간 초과 (2000걸음)
→ 같은 자리를 뱅뱅 돌다가 포기

메타코딩은 화면을 멍하니 바라봤습니다. 자동차가 한 칸 앞에 목적지가 있는데도 계속 제자리에서 돌고 있었습니다. 마치 미로에 갇힌 생쥐처럼.

💥 왜 실패했을까?

원인은 ***"눈앞만 보는 한계"***였습니다.

자동차의 사고 과정:

"목적지가 오른쪽이니까 오른쪽으로!"

→ 오른쪽은 벽!

"그럼 아래로 가자!"

→ 아래도 벽!

"그럼 위로!"

→ 위도 벽!

"다시 오른쪽!" (처음으로 돌아옴)

→ 오른쪽은 벽!

... (무한 반복)

이것이 **국소 최적화 함정(Local Optima)** 입니다. "지금 이 순간 가장 좋아 보이는 선택"만 반복하다 보면, 정작 전체적으로 봤을 때 최선의 길을 찾지 못하는 것이죠.

실생활에서도 이런 일은 흔합니다. 매일 같은 출근길만 고집하는 사람이 있습니다. 어느 날 공사 때문에 막혔는데, "원래 길이 제일 빠르니까"라며 1시간을 기다립니다. 돌아가면 20분이면 도착하는데 말이죠.

하지만 진짜 문제는 따로 있었습니다.

❌ if 문 (규칙 기반):

"막히면 이쪽, 막히면 저쪽"

→ 1000번 실패해도 1001번째도 똑같이 실패!

→ 경험으로부터 배우지 않음!

✅ AI (강화학습):

"이 상황에서 오른쪽으로 갔더니 벽이었어."

"다음엔 위로 먼저 가봐야겠어."

→ 실패할수록 더 잘해짐! 점점 발전!

규칙 기반 시스템에는 **"기억"이 없습니다**.昨天的 실패를 오늘의 교훈으로 바꿀 수 없습니다. 하지만 AI는 다릅니다. AI는 실패를 기억하고, 그 기억에서 패턴을 찾아내고, 다음에는 더 나은 선택을 합니다.

🧠 메타코딩의 깨달음

메타코딩은 노트에 적었습니다.

"규칙만으로는 한계가 있다. 복잡한 환경에서는 스스로 배우는 AI가 필요하다."

사람도 마찬가지입니다. 처음 운전을 배울 때 "빨간불이면 멈춰라"는 규칙만으로는 복잡한 도로를 달릴 수 없습니다. 수백 번의 경험을 쌓으면서 "이런 상황에서는 이렇게"라는 감각이 생기는 것처럼, AI도 경험이 필요합니다.

"좋아, 이제 진짜 AI를 만들어보자!"

하지만 AI를 만들기 전에, 먼저 결정해야 할 것이 있었습니다.

AI에게 세상을 보여주는 방법

본격적으로 AI를 만들기 전에, 메타코딩은 한 가지 중요한 질문과 마주했습니다.

🤖 "AI에게 뭘 알려줘야 하지?"

"전체 맵 정보를 다 줘야 할까?"

만약 전체 맵을 통째로 AI에게 보여준다면 어떻게 될까요? AI는 그 맵을 사진 찍듯 외워버립니다. "왼쪽에서 세 번째 칸에서 오른쪽으로 꺾어"라고 외우는 거죠. 당연히 맵이 바뀌면 아무것도 못 합니다.

마치 서울에서 부산까지 가는 길을 "신호등 237개째에서 좌회전"이라고 외우는 것과 같습니다. 도로 공사라도 하면 바로 길을 잃어버리죠.

"나침반처럼 방향만 알려주면 어떨까? '지금 목적지가 저쪽이야. 그리고 바로 주변에 벽이 있어.' 이 정도면 충분하지 않을까?"

이 아이디어가 핵심이었습니다.

🏠 AI가 받는 정보: 상태 벡터 (State Vector)

상태 벡터란 현재 상황을 숫자들의 묶음으로 표현한 것입니다. AI는 글자나 그림이 아니라, **오직 숫자만** 이해합니다. 그래서 "지금 앞에 벽이 있어"를 "1"로, "앞에 벽이 없어"를 "0"으로 변환해서 알려줍니다.

우리 AI는 총 **11개의 숫자**를 받습니다:

AI가 보는 세상 (총 11개 숫자):

[왼위벽]	[위 벽]	[오른위벽]	
①=0	②=1	③=0	← 벽이면 1, 빈칸이면 0
[왼쪽벽]		[오른쪽벽]	
④=0		⑤=0	
[왼아래벽]	[아래벽]	[오른아래벽]	
⑥=1	⑦=0	⑧=0	
⑨ 현재 방향: 오른쪽(→)			
⑩ 목적지 X 방향: +0.6 (오른쪽으로 60% 거리)			
⑪ 목적지 Y 방향: +0.4 (아래쪽으로 40% 거리)			

왜 하필 11개일까요?

번호	정보	값 예시	역할
----	----	------	----

번호	정보	값 예시	역할
①~③	주변 8칸 벽 유무	0 또는 1	"지금 주변이 어떤가?"
⑨	현재 바라보는 방향	0~3	"나는 어디를 보고 있나?"
⑩	목적지 X 방향 거리	-1.0 ~ +1.0	"목적지가 왼쪽? 오른쪽?"
⑪	목적지 Y 방향 거리	-1.0 ~ +1.0	"목적지가 위? 아래?"

이 11개 숫자가 AI의 "눈" 입니다. 전체 지도가 아니라, "지금 내 주변"과 "목적지 방향"만 아는 것이죠. 사람이 낯선 골목에서 나침반과 눈을 가지고 길을 찾는 것과 비슷합니다.

🎯 보상 시스템: AI에게 칭찬과 꾸중하기

정보만 준다고 AI가 배우는 건 아닙니다. "**잘했어!**" (보상) 와 "**안 돼!**" (패널티) 가 있어야 무엇이 좋고 나쁜지 알 수 있습니다.

강아지를 훈련시킬 때를 생각해 보세요. "앉아!"를 했을 때 간식을 주면, 강아지는 "앉으면 좋은 일이 생기는구나!"를 배웁니다. 쓰레기통을 뒤지면 혼나니까, "쓰레기통은 건드리면 안 되는구나"를 배우죠.

AI도 마찬가지입니다:

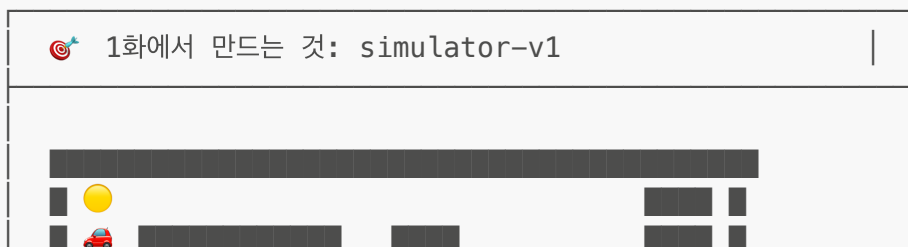
```
if 목적지 도달:
    reward = +100 # "완벽해! 바로 이거야!"
elif 목적지에 가까워짐:
    reward = +0.5 # "좋아, 방향은 맞아!"
elif 벽에 충돌:
    reward = -10 # "앗! 벽이야, 조심해!"
elif 시간 초과:
    reward = -50 # "너무 오래 걸렸어!"
```

이걸 **보상 함수(Reward Function)** 라고 합니다. AI의 선생님 같은 존재입니다.

여기서 중요한 점이 있습니다. 보상을 어떻게 설계하느냐에 따라 AI의 행동이 완전히 달라집니다. 예를 들어 "벽에 부딪히면 -10"이 아니라 "-1000"으로 했다면? AI는 벽이 너무 무서워서 아예 움직이지 않으려 할 수도 있습니다. 반대로 "-1"로 했다면? "벽에 좀 부딪혀도 괜찮지"라며 벽에 마구 부딪힐 수 있습니다.

좋은 보상 함수를 설계하는 것은 AI에서 매우 중요한 기술입니다. 마치 좋은 선생님이 적절한 칭찬과 꾸중의 균형을 잡는 것처럼요.

1화: 첫 번째 AI, 첫 번째 함정 (v1)



특징: DQN으로 처음 학습! (500 에피소드)
 문제: 항상 같은 경로만 (과적합!)
 성공률: 80% (같은 맵에서만!)

드디어 AI를 만들다!

메타코딩은 팔을 걷어붙이고 코드를 짜기 시작했습니다. 검색을 거듭할수록 **DQN(Deep Q-Network)** 이 가장 적합하다는 확신이 들었습니다.

"DQN... 이름은 어렵지만, 핵심은 간단해."

한 줄로 요약하면 이렇습니다:

"AI에게 '이 상황에서 이 행동을 하면 얼마나 좋을까?'를 계산하는 뇌를 만들어주는 것."

이 "뇌"가 바로 **신경망(Neural Network)** 입니다.

신경망(Neural Network): AI의 뇌

"사람의 뇌를 수학으로 흉내 낸 것"

사람의 뇌에는 약 860억 개의 **뉴런(신경세포)** 이 있습니다. 이 뉴런들은 서로 연결되어 있고, 전기 신호를 주고받으며 "생각"을 합니다. 눈으로 들어온 빛 신호가 뉴런을 거치면서 "이건 고양이다"라는 판단이 되는 거죠.

신경망은 이 과정을 흉내 냅니다. 물론 860억 개는 무리고, 우리 AI는 소박하게 **78개의 뉴런**으로 시작합니다.

[우리 AI의 신경망 구조]

입력층 (11개)

- ① 왼쪽벽? = 0
- ② 위 벽? = 1
- ③ 오른쪽벽? = 0
- ④ 왼쪽벽? = 0
- ⑤ 오른쪽벽? = 0
- ⑥ 왼쪽벽? = 1
- ⑦ 아래벽? = 0
- ⑧ 오른쪽벽? = 0
- ⑨ 현재방향 = 1(→)
- ⑩ 목적지dx = +0.6
- ⑪ 목적지dy = +0.4

은닉층 (64개 뉴런)

64개 뉴런
이 각각
계산 중...

↑
각 뉴런이 입력값에
가중치를 곱하고 더해서
최종 Q값을 만듦

출력층 (3개)

직진 Q = 8.5
우회전 Q = 3.2
좌회전 Q = 2.1

↑
Q값이 가장 높은
행동을 선택!

11개 숫자를 넣으면, 자동으로 "직진/우회전/좌회전" 중 어느 것이 가장 좋은지 **Q값**으로 알려줍니다.

뉴런 하나는 어떻게 계산할까?

은닉층의 뉴런 하나가 하는 일을 들여다봅시다. 생각보다 단순합니다.

[뉴런 하나의 계산 과정 - 3단계]

1단계: 각 입력에 "가중치(weight)"를 곱한다

입력값	가중치(w)	결과
	×	=
① 0 (벽없음)	× 0.3	= 0
② 1 (벽있음)	× 0.8	= 0.8
③ 0	× 0.1	= 0
④ 0	× 0.5	= 0
⑤ 0	× 0.2	= 0
⑥ 1 (벽있음)	× 0.7	= 0.7
⑦ 0	× 0.4	= 0
⑧ 0	× 0.1	= 0
⑨ 1 (오른쪽)	× 0.6	= 0.6
⑩ 0.6	× 0.9	= 0.54
⑪ 0.4	× 0.3	= 0.12

2단계: 전부 더하고, 편향(bias)도 더한다

$$\begin{aligned} \text{합계} &= 0 + 0.8 + 0 + 0 + 0 + 0.7 + 0 + 0 \\ &\quad + 0.6 + 0.54 + 0.12 + \text{편향}(0.1) \\ &= 2.86 \end{aligned}$$

3단계: 활성화 함수를 통과시킨다 (ReLU)

$$\text{ReLU}(2.86) = 2.86 \quad \checkmark \quad (\text{양수니까 그대로!})$$

이걸 수식으로 쓰면:

$$\text{뉴런 출력} = \text{ReLU}(\sum(\text{입력} \times \text{가중치}) + \text{편향})$$

가중치(weight) 가 바로 AI가 학습하는 것입니다! 처음에는 가중치가 랜덤이라 엉터리 값이 나옵니다. 하지만 학습을 반복하면서 "이 가중치를 이렇게 바꾸면 더 정확해지겠다"를 발견하고, 조금씩 수정해나갑니다.

64개 뉴런이 각각 이런 계산을 하고, 그 결과들이 다시 합쳐져서 최종 Q값 3개가 만들어집니다.

활성화 함수: ReLU와 Sigmoid

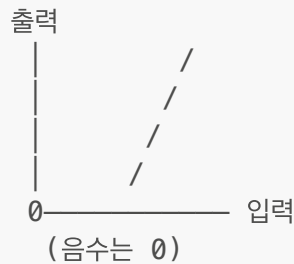
뉴런 계산의 마지막에 **활성화 함수**를 통과시킨다고 했죠. 이게 왜 필요할까요?

활성화 함수가 없으면 신경망이 아무리 깊어도 결국 "커다란 곱셈+덧셈" 하나와 같아집니다. 1층 건물이든 100층 건물이든 같은 높이가 되는 셈이죠. 활성화 함수가 있어야 **비선형** 문제, 즉 직선 하나로는 풀 수 없는 복잡한 문제(복잡한 미로 같은!)를 풀 수 있습니다.

[두 가지 대표적인 활성화 함수]

① ReLU (Rectified Linear Unit) – 우리 AI가 사용!

규칙: 음수면 0, 양수면 그대로 통과



예시: $\text{ReLU}(2.86) = 2.86$

$\text{ReLU}(-1.3) = 0$

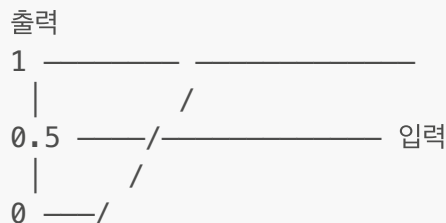
$\text{ReLU}(0) = 0$

장점: 계산이 매우 빠름!

현재 딥러닝에서 가장 많이 쓰이는 활성화 함수

② Sigmoid (시그모이드)

규칙: 어떤 값이든 0~1 사이로 부드럽게 압축



예시: $\text{Sigmoid}(10) = 0.99$ (거의 1)

$\text{Sigmoid}(0) = 0.5$ (딱 중간)

$\text{Sigmoid}(-10) = 0.01$ (거의 0)

장점: 확률을 표현하기 좋음!

예: "목적지 도달 확률 73%" → 0.73

우리 v1~v5에서는 ReLU를 사용합니다.

은닉층에서 빠르게 계산하기 위해서죠.

Sigmoid는 주로 최종 출력에서 "확률"을 표현할 때 씁니다.

💡 **비유로 이해하기:** ReLU는 "나쁜 신호 차단 필터"입니다. 뉴런이 "이 정보는 쓸모없어"라고 판단하면(음수) 완전히 차단(0)하고, "이건 유용해!"라고 판단하면(양수) 그대로 통과시킵니다.

🧠 Q값과 DQN: AI의 판단 기준

Q값(Q-value)은 "이 상황에서 이 행동을 하면 **앞으로** 얼마나 좋을까?"를 나타내는 점수입니다.

여기서 중요한 건 "앞으로"라는 단어입니다. 지금 당장의 보상만이 아니라, **미래에 받을 보상까지 고려한 점수**입니다. 이것이 0 화의 if 규칙과 근본적으로 다른 점입니다.

[예시: 현재 상황 - 목적지가 오른쪽 아래, 앞길이 열려 있음]

Q(현재 상황, 직진) = 8.5 ← 가장 높음! → 이 행동 선택!

Q(현재 상황, 우회전) = 3.2

Q(현재 상황, 좌회전) = 2.1

AI는 Q값이 가장 높은 "직진"을 선택!

처음에는 이 Q값이 엉터리입니다. 가중치가 랜덤이니까요. 그러다 실패를 경험하면서 "이 상황에서 직진하면 벽이다 → Q값을 낮추자"를 배우고, 성공을 경험하면서 "이 상황에서 우회전하면 목적지에 가까워진다 → Q값을 올리자"를 배웁니다.

이 과정을 수백, 수천 번 반복하면 **Q값이 점점 정확해집니다. 이것이 DQN의 핵심입니다!**

DQN = Deep(깊은 신경망으로) + Q(Q값을) + Network(계산하는 네트워크)

🎮 학습 시작!

```
cd simulator-v1
python train.py
```

드디어 화면에 작은 파란 자동차가 나타났습니다. 메타코딩은 심장이 두근거렸습니다. 내가 만든 AI가 처음으로 세상에 나온 순간!

하지만 처음에는... 처참했습니다.

처음 100번 (Episode 0~100): 아무것도 모르는 갓난아이

[Episode 47 - 초보 단계]



Episode: 47 | Score: -10.0 | ϵ : 0.79

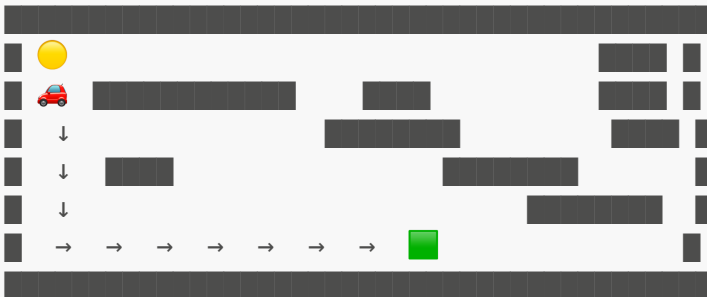
→ 랜덤하게 움직이다 벽에 충돌!

"어... 이게 AI야?"

자동차가 술 취한 사람처럼 비틀거리며 벽에 쿵쿵 부딪혔습니다. 하지만 이건 정상입니다! ϵ (엡실론) = 0.79라는 숫자를 보세요. 이것은 "79%의 확률로 랜덤하게 행동한다"는 뜻입니다. 아직은 AI가 세상을 탐험하는 단계입니다. 마치 아기가 처음 걸음마를 시작할 때처럼, 넘어지고 부딪히면서 세상을 알아가는 중이에요.

중간 200번 (Episode 100~300): 원가 감을 잡기 시작

[Episode 213 - 학습 중]



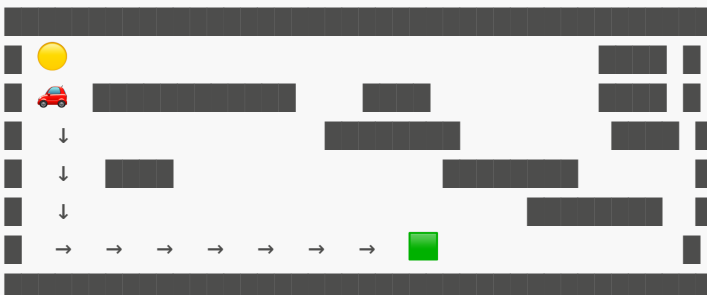
Episode: 213 | Score: 52.3 | ϵ : 0.34

→ 패턴을 찾기 시작! 아래로 가다가 오른쪽으로!

뭔가 달라졌습니다! 자동차가 의미 있는 경로를 따라 움직이기 시작했습니다. 아직 완벽하지 않지만, "아래로 가다가 오른쪽으로"라는 패턴을 발견한 것이죠. ϵ 도 0.34로 줄어들었습니다. "34%만 탐험, 66%는 배운 대로 행동." 점점 배운 것을 활용하기 시작한 겁니다.

마지막 200번 (Episode 300~500): 완성!

[Episode 487 - 완성 단계]



Episode: 487 | Score: 98.2 | ϵ : 0.08

→ 매번 같은 경로로 성공! 성공률 80%!

"와! 진짜 된다! AI가 스스로 길을 찾았어!"

메타코딩은 의자에서 벌떡 일어나 주먹을 불끈 쥐었습니다. 500번의 시행착오를 통해 AI가 스스로 최적 경로를 발견한 것입니다. 이 기쁨은 직접 만들어본 사람만 알 수 있습니다.

 그런데... 밤늦게까지 보다 보니

흥분이 가라앉고 나서, 메타코딩은 이상한 점을 발견했습니다.

자동차가 **항상 똑같은 경로**로만 움직였습니다.

매번 이것만 반복:
시작점(●) → 아래로 꺾 → 오른쪽으로 꺾 → 목적지(■)

"뭐, 도착하니깐 괜찮지? 다른 길이 있어도 이 길로 가면 되잖아."

메타코딩은 스스로를 설득하고 잠자리에 들었습니다.

🔥 다음날 — 팀장님 앞에서 대참사

"자, 팀장님! 보여드리겠습니다!"

자신 있게 프로그램을 실행했습니다. 자동차가 익숙한 경로로 목적지에 도달했습니다. 팀장님도 "오!" 하며 고개를 끄덕였습니다.

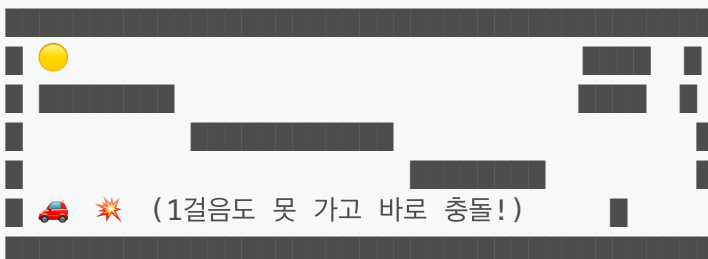
"좋네요! 그런데 메타코딩 씨, 청소 로봇은 여러 집에서 작동해야 하잖아요. 이 맵이 아닌 **다른 맵**에서도 잘 되나요?"

팀장님이 설정을 바꿨습니다.

```
env = GridEnvironment(random_map=True) # 새로운 맵!
```

결과:

[새로운 맵에서 v1 실행]



→ 새로운 맵에서는 완전히 무력!

메타코딩의 얼굴이 새빨개졌습니다.

팀장님: "메타코딩 씨... 이 맵에서만 작동하는 거예요?"

메타코딩: "아... 네... 죄송합니다..."

🤔 무엇이 문제였을까?

집에 돌아와서 메타코딩은 문제를 분석했습니다.

문제 1: 항상 같은 방향에서 시작

```
# car.py
self.direction = 0 # 항상 위쪽만 보고 시작!
```

항상 위쪽을 보고 시작하니까, AI는 "위에서 시작하면 아래로 내려가라"만 배운 것입니다. 오른쪽이나 왼쪽을 보고 시작하면 어떻게 해야 할지 전혀 모릅니다.

문제 2: 항상 같은 맵에서 학습

```
# train.py
env = GridEnvironment() # 항상 같은 맵!
```

한 집의 구조만 외운 청소부가 다른 집에 가면? 당연히 길을 잃습니다.

문제 3: 너무 빨리 탐험을 멈춤

```
self.epsilon_decay = 0.995 # 빠르게 탐험 줄임
```

아직 세상을 충분히 탐험하지 않았는데 "이제 아는 길로만 하자!"라며 탐험을 그만둔 것입니다.

이 세 가지 문제가 합쳐진 결과가 바로 **과적합(Overfitting)**입니다.

[과적합을 한 마디로]

학생이 수학 시험을 준비했습니다.
문제집 1번 문제를 1000번 풀었습니다.
1번 문제는 눈 감고도 풀 수 있게 됐어요!

시험 날, 2번 문제가 나왔습니다.
→ 완전히 틀렸어요! ❌

이게 과적합입니다.
"연습 문제는 완벽한데, 새 문제는 전혀 못 푸는 것."

반대 개념은 **일반화(Generalization)**입니다. "어떤 문제가 나와도 풀 수 있는 진짜 실력." 우리 AI가 가져야 할 능력이지요.

메타코딩은 노트에 크게 적었습니다.

"v1의 교훈: 과적합은 가짜 성공이다. 진짜 실력은 새로운 상황에서 드러난다."

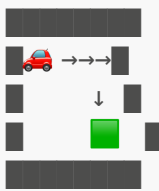
"내일부터 다시 시작이다!"

2화: 실패에서 배우는 법 (v2)

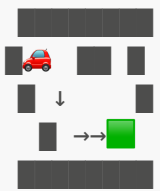
🎯 2화에서 만드는 것: simulator-v2

20개 맵에서 돌아가며 학습!

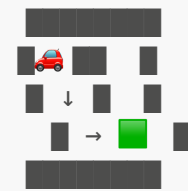
맵 #0



맵 #7



맵 #15



특징: 3000 에피소드, 매 150번마다 맵 교체
성공률: 70% (처음 보는 맵에서도!)

📅 2주차 월요일 — 다시 시작

주말 내내 v1의 실패가 머릿속에서 떠나지 않았습니다. 월요일 아침, 메타코딩은 커피를 한 잔 크게 내리고 노트를 펼쳤습니다.

"문제는 세 가지. 같은 방향, 같은 맵, 빠른 탐험 종료. 하나씩 해결하자."

🔧 해결책 1: 랜덤 방향으로 시작

```
# v1 (항상 위쪽을 봄)
self.direction = 0

# v2 (랜덤하게 시작!)
self.direction = random.randint(0, 3)
# 0: 위, 1: 오른쪽, 2: 아래, 3: 왼쪽
```

이제 매 에피소드마다 다른 방향으로 시작합니다.

```
Episode 1: 위쪽 보고 시작 → 위로 가는 법 배움
Episode 2: 오른쪽 보고 시작 → 오른쪽으로 가는 법 배움
Episode 3: 아래쪽 보고 시작 → 아래로 돌아가는 법 배움
Episode 4: 왼쪽 보고 시작 → 왼쪽에서 시작하는 법 배움
...
→ 어떤 방향에서 시작해도 대응 가능! ✅
```

🔧 해결책 2: 여러 맵에서 학습

```
# 150 에피소드마다 맵 변경
if episode % 150 == 0:
    map_id = random.randint(0, 19) # 0~19번 맵 중 하나
    env.reset_map(map_id)
```

20개의 다양한 맵을 돌아가며 학습합니다.

맵 0 (빈 맵) : 직진하는 법 배움
 맵 5 (중간 맵) : 장애물 하나 피하는 법 배움
 맵 10 (복잡한 맵) : 복잡한 미로 탈출하는 법 배움
 맵 15 (좁은 통로) : 좁은 길을 따라가는 법 배움
 → 어떤 환경에서도 어느 정도 작동! ✅

여러 문제집을 번갈아 풀면 진짜 수학 실력이 느는 것처럼, AI도 다양한 환경에서 학습해야 진짜 실력이 붙습니다.

🧠 경사하강법(Gradient Descent): AI가 실수에서 배우는 원리

"AI가 틀릴 때마다, 어떻게 스스로를 고치는가?"

AI가 여러 맵에서 학습을 할 때, 내부에서는 어떤 일이 벌어질까요? 바로 **경사하강법(Gradient Descent)** 이 작동합니다. 이것은 AI 학습의 핵심 중의 핵심입니다.

🏔️ 비유: 안개 속 산에서 가장 낮은 곳 찾기

여러 봉우리와 골짜기가 있는 복잡한 산을 상상해보세요. 짙은 안개가 끼어서 주변이 하나도 안 보입니다. 전체 지도도 없습니다. 이 상황에서 ****가장 낮은 곳(골짜기)****을 찾아야 합니다.

어떻게 하면 될까요?

전략: 발로 경사를 느끼면서 내려가기!

1. 발을 앞뒤좌우로 내밀어서 경사를 느낀다 → "기울기 계산"
2. 가장 가파르게 내려가는 방향으로 한 걸음 → "가중치 업데이트"
3. 다시 경사를 느끼고, 또 한 걸음 → "반복!"
4. 더 이상 내려갈 곳이 없으면 멈춘다 → "수렴"

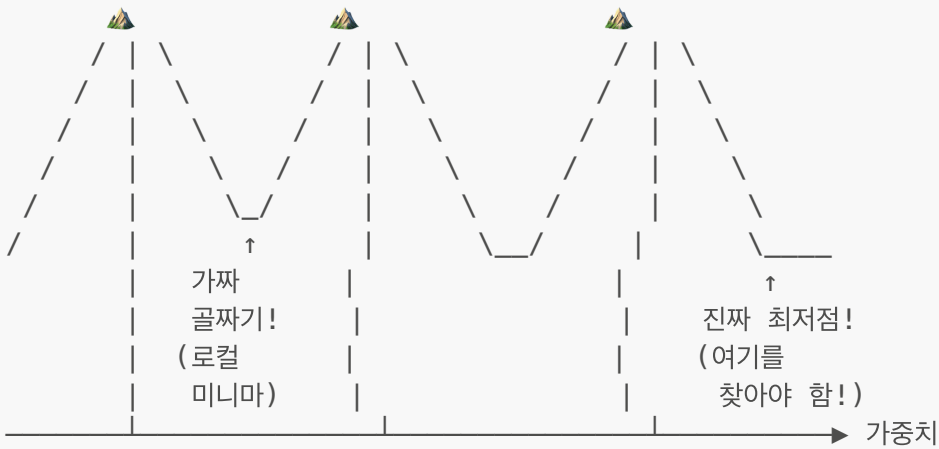
이것이 경사하강법의 전부입니다! 생각보다 간단하죠?

여기서 **"경사를 느끼는 것"이 바로 미분(기울기 계산)** 입니다. 수학의 미분이 여기서 쓰입니다. AI가 "내가 지금 있는 위치에서 어느 방향으로 가면 오차가 줄어들까?"를 계산하는 것이 미분이에요.

🏔️ 하지만 함정이 있습니다: 로컬 미니마!

산에는 여러 골짜기가 있습니다. 발로만 느끼면서 내려가다 보면 *****가짜 골짜기*****에 빠질 수 있습니다.

손실(Loss) - 높을수록 AI가 많이 틀린 것!

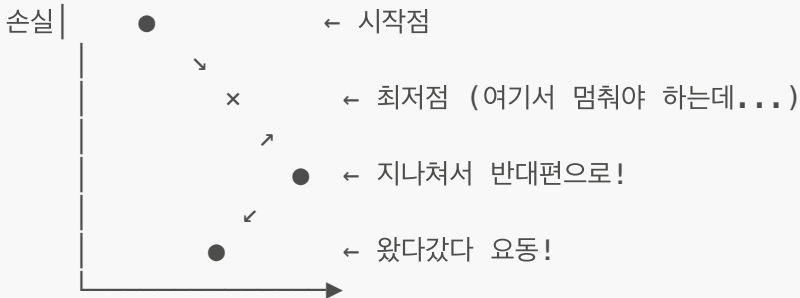


"가짜 골짜기(로컬 미니마)"에 도달하면, 발로 느껴보는 모든 방향이 올라가는 것 같습니다. "여기가 가장 낮은 곳이구나!"라고 착각하죠. 하지만 실제로는 더 낮은 곳(진짜 최저점, 글로벌 미니마)이 멀리 있습니다.

🔧 보폭(Learning Rate)이 중요한 이유

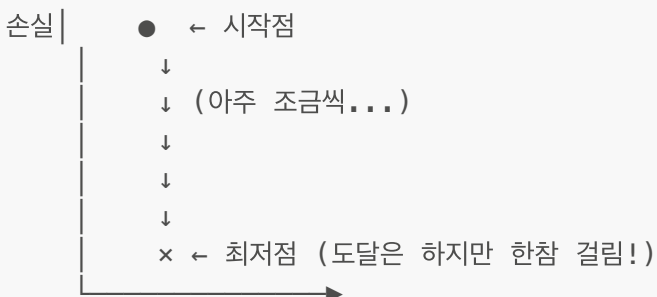
안개 속에서 한 걸음의 크기를 **보폭(Learning Rate)** 이라고 합니다. 이 보폭이 학습의 운명을 결정합니다.

[보폭이 너무 클 때 - 최저점을 지나쳐버림!]



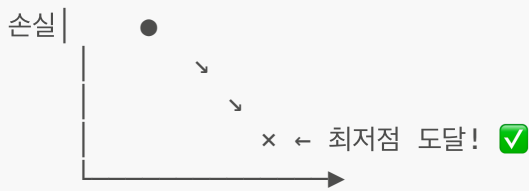
큰 보폭으로 성큼성큼 내려갔더니
최저점을 지나쳐서 반대편으로 넘어감!
계속 왔다갔다 하면서 수렴하지 못함.

[보폭이 너무 작을 때 - 도착은 하지만 너무 느림!]



계다가 로컬 미니마에 빠지면 빠져나올 수도 없음!
작은 보폭으로는 함정에서 탈출할 에너지가 없어서.

[딱 맞는 보폭 - 빠르고 정확하게!]



🎯 최적의 전략: 처음엔 크게, 나중엔 작게!

가장 똑똑한 방법은 보폭을 점점 줄이는 것입니다.

1단계: 큰 보폭으로 전체를 훑는다

"이 산 어디쯤에 가장 낮은 곳이 있을까?"

→ 성큼성큼 걸어서 대략적인 위치를 파악!

→ 로컬 미니마도 큰 보폭으로 뛰어넘을 수 있음!

2단계: 작은 보폭으로 정밀하게 찾는다

"이 근처가 가장 낮은 것 같은데, 정확히 어디지?"

→ 조심조심 걸어서 진짜 최저점을 찾음!

→ 빠르면서도 정확! ✅

이 기법을 Learning Rate Scheduling이라 하며,
4화에서 자세히 다룹니다!

12 34 실제 AI에서 경사하강법은 이렇게 동작합니다

추상적인 산 이야기에서 구체적인 숫자로 내려와 봅시다.

[v2에서 한 번의 학습 과정 - 6단계]

1단계: AI가 Q값을 예측한다

현재 상황 → 신경망 통과 → Q(직진) = 7.0

"직진하면 7.0점짜리 좋은 결과가 나올 거야!"

2단계: 실제로 행동하고 결과를 확인한다

직진했더니 → 목적지에 가까워짐!

실제 보상 기반 올바른 Q값(정답): 10.0

3단계: 오차(Loss)를 계산한다

$$\begin{aligned}\text{오차} &= (\text{예측} - \text{정답})^2 \\ &= (7.0 - 10.0)^2 \\ &= (-3.0)^2 \\ &= 9.0\end{aligned}$$

→ "꽤 많이 틀렸네!"

이 오차를 손실(Loss) 이라고 합니다.
손실이 산의 높이입니다. 이걸 줄이는 게 목표!

4단계: 기울기(Gradient)를 계산한다 - 이것이 미분!

$$\begin{aligned}\text{기울기} &= 2 \times (\text{예측} - \text{정답}) \\ &= 2 \times (7.0 - 10.0) \\ &= -6.0\end{aligned}$$

기울기가 음수(-6.0) → "Q값을 올려야 해!"
기울기가 양수였다면 → "Q값을 내려야 해!"

이 기울기가 "경사"입니다.
"어느 방향으로 가중치를 수정하면 오차가 줄어드는가?"
를 알려주는 나침반 같은 존재!

5단계: 가중치를 수정한다 (한 걸음 내딛기!)

$$\begin{aligned}\text{학습률(보폭)} &= 0.1 \\ \text{수정량} &= \text{학습률} \times |\text{기울기}| = 0.1 \times 6.0 = 0.6\end{aligned}$$

$$\begin{aligned}\text{새 Q값} &= 7.0 + 0.6 = 7.6 \\ &\rightarrow "10.0\text{에 조금 가까워졌어!"}\end{aligned}$$

6단계: 이걸 3000번 반복!

$$7.0 \rightarrow 7.6 \rightarrow 8.1 \rightarrow 8.5 \rightarrow 9.0 \rightarrow 9.5 \rightarrow 9.8 \rightarrow 10.0$$

→ 점점 정답에 가까워짐! 

 **핵심 정리:** 기울기(Gradient)는 미분의 결과입니다. "오차를 줄이려면 가중치를 어느 방향으로, 얼마나 바꿔야 하는가?"를 알려주는 나침반이에요. 이 나침반을 따라 한 걸음씩(Learning Rate만큼) 이동하면, 오차가 점점 줄어듭니다.

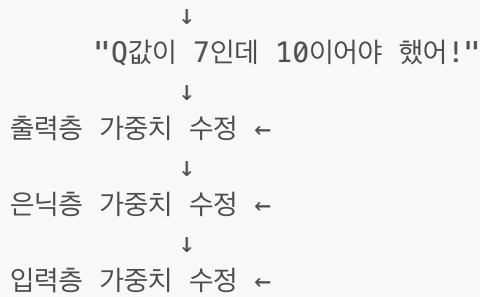
역전파(Backpropagation): 기울기를 전달하는 방법

방금 "기울기를 계산한다"고 했는데, 신경망에는 뉴런이 64개나 있습니다. 각 뉴런의 가중치를 어떻게 수정할까요?

역전파(Backpropagation) 는 출력에서 발생한 오차를 거꾸로 전달하면서, 각 뉴런이 "내가 얼마나 틀렸는지"를 계산하는 방법입니다.

[역전파 흐름]

입력 → 은닉층 → 출력 → 오차 발생!



마치 도미노를 거꾸로 세우는 것처럼,
오차의 "책임"을 뒤로 전파해서
각 가중치를 조금씩 수정합니다.

비유하면 이렇습니다. 축구에서 골을 넣지 못했을 때, 감독이 분석합니다. "슈팅이 빗나갔네(출력 오차). 왜? 패스가 정확하지 않았어(은닉층). 왜? 처음 볼 소유 때 위치 잡기가 나빴어(입력층)." 이렇게 원인을 거꾸로 추적하면서 각 선수(뉴런)에게 개선 방향을 알려주는 것이 역전파입니다.

🔧 해결책 3: 천천히 탐험 줄이기 (Epsilon 조절)

Epsilon(ϵ) 은 "랜덤하게 새로운 것을 시도할 확률"입니다.

$\epsilon = 1.0$ (100%): "모험가 모드! 완전 랜덤으로 움직여!"
 $\epsilon = 0.5$ (50%): "절반은 모험, 절반은 아는 길로"
 $\epsilon = 0.1$ (10%): "거의 아는 길, 가끔 새 시도"
 $\epsilon = 0.0$ (0%): "베테랑 모드! 배운 것만 활용!"

```
# v1 (너무 빨리 탐험을 줄임)
self.epsilon_decay = 0.995 # 금방 0에 가까워짐

# v2 (천천히 탐험을 줄임)
self.epsilon_decay = 0.998 # 오래 탐험 유지
```

왜 이게 중요할까요?

v1의 문제:

```
Episode 1 :  $\epsilon = 1.00$  (100% 탐험) "뭐든 해봐!"
Episode 50 :  $\epsilon = 0.60$  (60% 탐험) "좀 알아가는 중"
Episode 100 :  $\epsilon = 0.40$  (40% 탐험) "이제 아는 길로"
Episode 200 :  $\epsilon = 0.10$  (10% 탐험) "거의 아는 길만"
→ 세상의 10%도 안 봤는데 탐험을 포기!
→ 한 가지 길만 알고 멈춤!
```

v2의 개선:

Episode 100 : $\epsilon = 0.90$ (90% 탐험) "아직 많이 탐험"
 Episode 500 : $\epsilon = 0.60$ (60% 탐험) "여전히 탐험 중"
 Episode 1000 : $\epsilon = 0.40$ (40% 탐험) "배워가는 중"
 Episode 2000 : $\epsilon = 0.10$ (10% 탐험) "이제 활용 시작"
 → 충분히 탐험한 후에 활용! ✅

🧠 Exploration vs Exploitation: AI의 영원한 딜레마

"새로운 식당을 가볼까, 단골 식당에 갈까?"

이것은 AI 분야에서 가장 유명한 딜레마 중 하나입니다.

탐험 (Exploration):

"오늘은 저 골목 새 식당에 가볼까?"

- 더 맛있는 곳을 발견할 수도!
- 하지만 맛있는 곳일 수도 있음

활용 (Exploitation):

"역시 우리 동네 단골집이 최고야!"

- 안전한 선택, 확실한 만족
- 하지만 더 좋은 곳을 영영 모름

딜레마:

- 탐험만 하면 → 배운 것을 써먹지 못함
- 활용만 하면 → 더 좋은 방법을 놓침

최적의 균형:

처음엔 탐험을 많이, 나중엔 활용 위주
 (= Epsilon이 1.0에서 서서히 0.1로)

v1은 이 균형을 잘못 잡은 것입니다. 식당 세 군데만 가보고 "여기가 최고!"라며 다른 곳을 안 가본 셈이죠. v2에서는 충분히 다양한 식당을 돌아본 후에 단골집을 정합니다.

🔧 해결책 4 & 5: 추가 개선들

새로운 길 발견 보상:

```
# 처음 가보는 곳이면 보너스!
if (next_x, next_y) not in self.visited_positions:
    reward += 0.2
```

같은 길만 빙빙 도는 것을 방지합니다. 새로운 칸을 밟을 때마다 "잘했어!"라는 작은 보상을 줘서 탐험을 격려하는 것이죠.

학습 안정화 (Gradient Clipping):

```
# 한 번에 너무 크게 배우지 않도록 제한!
torch.nn.utils.clip_grad_norm_(self.policy_net.parameters(), 1.0)
```

Gradient Clipping 비유:

❌ 없을 때:
어떤 날은 기울기가 100, 어떤 날은 0.001
→ 학습이 불안정, 폭주할 위험!

✅ 있을 때:
기울기가 아무리 커도 1.0으로 제한
→ 매일 일정한 보폭으로 꾸준히 학습!

운전에 비유하면:
급커브에서 갑자기 핸들을 확 꺾으면 위험하잖아요.
"핸들은 최대 여기까지만!" 이라는 제한을 두는 것.

경험 리플레이(Experience Replay): AI의 복습 노트

v2에서 한 가지 더 중요한 기법이 있습니다. **경험 리플레이**입니다.

AI는 움직일 때마다 경험을 저장합니다.

경험 = (현재 상태, 행동, 보상, 다음 상태)

예시:
("벽이 오른쪽에 있고 목적지가 아래", "직진", +0.5, "벽이 없고 목적지가 아래")

이 경험들을 **경험 창고(Replay Buffer)**에 쌓아둡니다. 그리고 학습할 때 이 창고에서 랜덤으로 꺼내서 복습합니다.

[경험 리플레이 비유]

학생의 오답 노트 같은 것!

- 문제를 풀 때마다 오답 노트에 기록
"3번 문제 틀림. 정답은 이거."
- 복습 시간에 오답 노트에서 랜덤으로 꺼내서 풀기
→ 오래 전에 틀린 문제도 다시 복습!
- 왜 랜덤으로? 순서대로 복습하면
"앞부분은 잘 기억나는데 뒷부분은 까먹음"
→ 골고루 복습해야 전체 실력이 향상!

경험 창고 (Replay Buffer):

```

경험 1: 직진 → 벽 충돌 (-10)
경험 2: 우회전 → 목적지 접근 (+0.5)
경험 3: 직진 → 목적지 도달! (+100)
...
경험 10000: 좌회전 → 빈 공간 (0)
  
```

학습할 때: 여기서 32개를 랜덤으로 뽑아서 복습!
(이 32개를 "미니배치(Mini-batch)"라고 합니다)

💡 **미니배치(Mini-batch):** 경험 창고에서 한 번에 32개씩 뽑아서 학습하는 것을 **미니배치 학습**이라고 합니다. 왜 하나씩 안 하고 32개씩? 하나씩 하면 학습이 불안정하고(한 경험에 너무 좌우됨), 전부 다 하면 너무 느립니다. 32개는 "적당히 안정적이면서 적당히 빠른" 절충안입니다.

🎮 v2 실행 결과

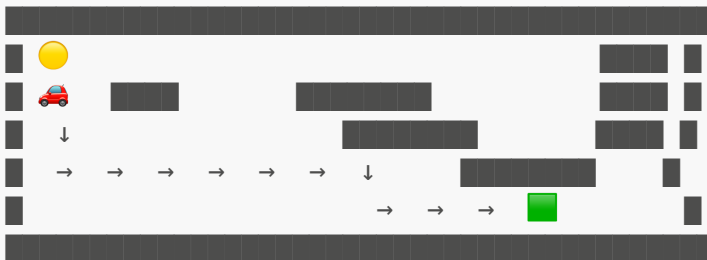
```

cd simulator-v2
python train.py
  
```

3000번의 에피소드 동안 20개 맵을 돌아가며 학습합니다.

[v2 - 학습 중 (Episode 1500)]

맵 #7에서 학습 중:



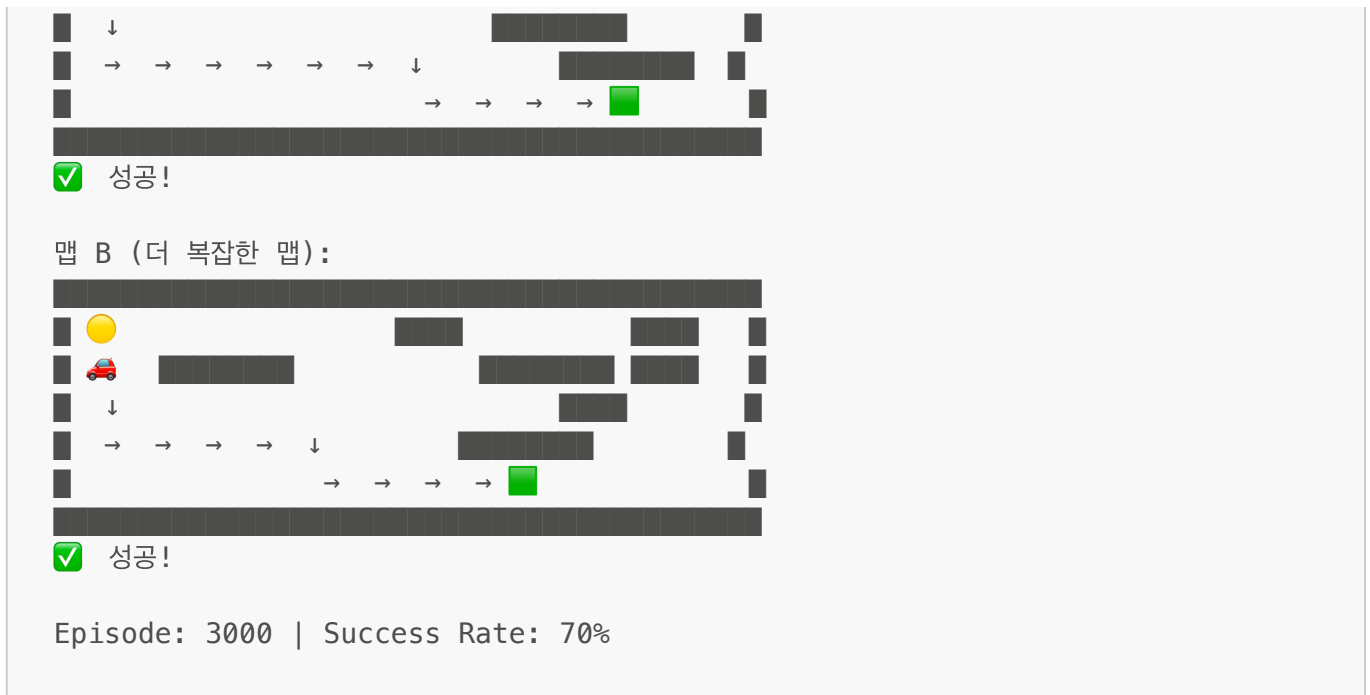
Episode: 1500 | Score: 5.8 | ϵ : 0.47 | Map: #7

3000번의 에피소드가 끝나자, 처음 보는 맵으로 테스트했습니다.

[v2 테스트 - 처음 보는 맵에서]

맵 A (본 적 없는 맵):





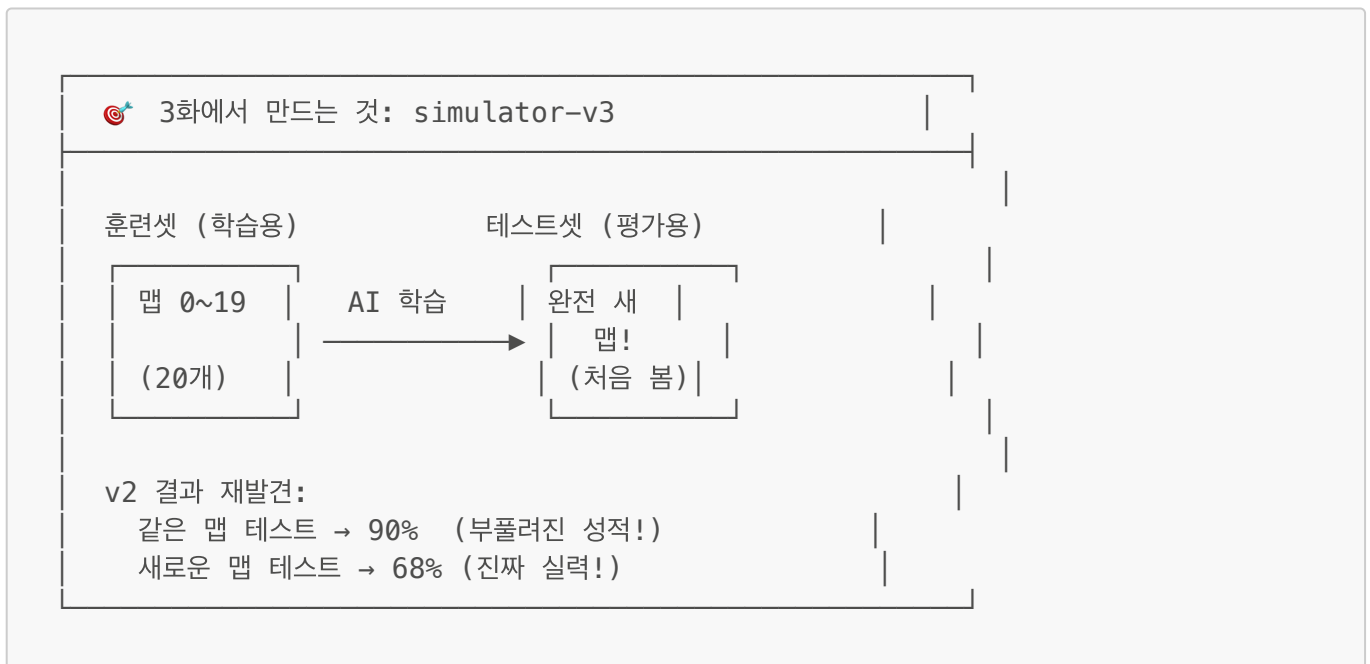
이번엔 팀장님 앞에서도 당당했습니다!

팀장님: "오! 이번엔 다른 맵에서도 잘 작동하네요!"

메타코딩은 속으로 v1의 실패에 감사했습니다. 그 실패가 없었다면 과적합의 무서움을 몰랐을 테니까요.

그런데...

3화: 진짜 실력을 증명하라 (v3)



17 3주차 - 선배의 한 마디

팀장님께 v2를 보여드리고 칭찬을 받은 그날 오후, 메타코딩은 회사 카페에서 커피를 내리고 있었습니다. 그때 뒤에서 익숙한 목소리가 들렸습니다.

"아, 메타코딩! 요즘 강화학습 프로젝트 하고 있다면서?"

김선배였습니다. 3년차 AI 개발자로, 회사에서 머신러닝 관련 질문이 생기면 모두가 찾아가는 사람이었습니다.

"네, v2까지 만들었어요! 이제 다른 맵에서도 70%나 성공해요!"

메타코딩은 자랑스럽게 대답했습니다.

"70%요? 괜찮네요." 김선배가 커피를 받아들며 물었습니다. "근데 그 70%, 어떤 맵으로 테스트한 거예요?"

"20개 맵으로 학습하고, 그 20개 중에서 테스트했어요."

커피를 마시던 김선배가 멈췄습니다. 눈이 동그아졌습니다.

"잠깐. 학습한 맵이랑 테스트한 맵이 **같다고요?**"

"네... 왜요?"

김선배가 커피를 테이블에 내려놓으며 의자를 끌어당겼습니다.

"메타코딩 씨, 앉아봐요. 이건 중요한 얘데요."

메타코딩은 갑자기 불안해지며 맞은편에 앉았습니다.

"이렇게 생각해봐요. 학생한테 수학 문제 20개를 줬어요. 그걸로 열심히 공부했죠. 그리고 시험 날, **그 20개 문제가 그대로** 나왔어요. 100점 받았어요. 그게 진짜 수학 실력이에요?"

"아... 아니죠. 그건 문제를 외운 거죠."

"맞아요! 지금 메타코딩 씨의 AI가 딱 그 상황이에요."

김선배가 냅킨에 그림을 그리기 시작했습니다.

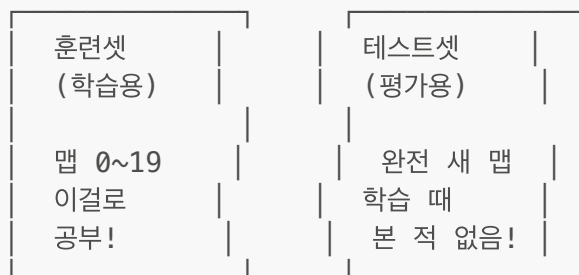
"학습할 때 20개 맵을 봤잖아요. 그 20개 맵의 벽 위치, 통로 패턴을 AI가 어느 정도 '외웠을' 수 있어요. 그 맵으로 테스트하면 당연히 점수가 높죠. 하지만 그게 진짜 길 찾기 실력인지, 아니면 맵을 외운 건지 **구분할 수가 없어요.**"

메타코딩의 가슴이 철렁 내려앉았습니다. 또 과적합?

"그럼... 어떻게 해야 해요?"

"간단해요." 김선배가 냅킨에 선을 하나 그었습니다.

데이터를 두 덩어리로 나누세요!



↑
여기서 배우고

↑
여기서 시험!

"학습에 쓰는 데이터를 **훈련셋(Training Set)**, 평가에 쓰는 데이터를 **테스트셋(Test Set)** 이라고 해요. 이 둘은 반드시 분리해야 합니다. 시험 문제가 연습 문제랑 같으면 안 되잖아요."

훈련셋 vs 테스트셋 — AI에서 가장 중요한 원칙

메타코딩은 이 개념을 노트에 크게 적었습니다.

[학교 시험에 비유]

연습 문제집 (훈련셋):

```
1번: 2 + 3 = ?
2번: 5 × 4 = ?
...
20번: 8 - 2 = ?
```

이걸로 공부

시험지 (테스트셋):

—AI—
학습

```
21번: 7 + 8 = ?
22번: 3 × 9 = ?
...
30번: 6 ÷ 2 = ?
```

처음 보는 문제!
이걸로 진짜 실력 평가!

이 원칙은 AI 분야 전체에서 통용됩니다. 이미지 인식 AI도, 번역 AI도, 추천 AI도, 모두 **훈련셋**과 **테스트셋**을 분리합니다. 분리하지 않으면 AI의 진짜 실력을 절대 알 수 없습니다.

v3의 수정

수정 자체는 놀랍도록 간단했습니다.

```
# v2: 학습한 맵으로 테스트 (❌ 잘못됨!)
test_env = GridEnvironment()

# v3: 완전히 새로운 맵으로 테스트 (✅ 올바름!)
test_env = GridEnvironment(random_map=True) # 처음 보는 맵!

print("△ 학습에 사용하지 않은 새로운 맵에서 테스트합니다!")
```

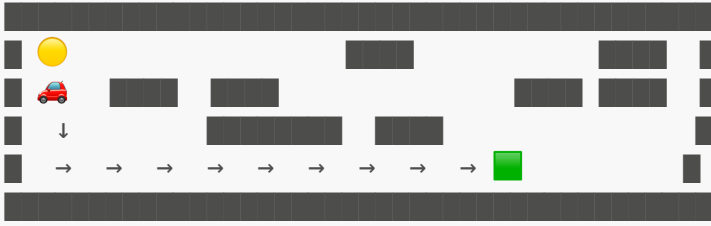
코드 한 줄의 변경이지만, 그 의미는 거대했습니다. "거울 앞에서 연습하던 것"을 "진짜 무대에 세우는 것"으로 바꾼 것이니까요.

충격적인 결과

```
cd simulator-v3
python main.py
```

△ 학습에 사용하지 않은 새로운 맵에서 테스트합니다!

[완전히 새로운 맵에서 v3 테스트]



Success Rate: 68% ← 진짜 실력!

v2에서 90%라고 믿었던 성공률이 사실은 ****68%****였습니다.

"22%나 차이가 나네..."

메타코딩은 충격을 받았습니다. 하지만 동시에 중요한 것을 깨달았습니다. **모르고 있는 것보다, 진실을 아는 게 훨씬 낫다.**

v2에서 착각한 것:

"90% 성공! 우리 AI 대단하다!"

v3에서 발견한 진실:

"사실 68%. 22%는 맵을 외운 것."

하지만!

68%도 대단한 거예요!

v0은 복잡한 맵에서 0%였잖아요.

적어도 이제 '진짜 실력'을 측정할 수 있게 됐어요.

팀장님께 솔직하게 보고

다음날 메타코딩은 팀장님께 솔직하게 보고했습니다.

"팀장님, v2의 90%는 정확하지 않았습니다. 진짜 새로운 환경에서 테스트하니 68%였습니다."

팀장님이 잠시 침묵했습니다. 메타코딩의 심장이 빠르게 뛰었습니다.

"메타코딩 씨, 이게 오히려 좋아요."

팀장님이 고개를 끄덕이며 말했습니다.

"68%가 진짜 실력이고, 이제 뭘 개선해야 하는지 방향이 생겼잖아요. 90%라고 착각하고 있었으면 어떻게 됐겠어요? 제품 출시하고 나서 엉망인 걸 발견했겠죠. 그게 더 큰 재앙이에요."

메타코딩은 고개를 끄덕였습니다. 정직한 측정이 있어야 정확한 개선이 가능하다.

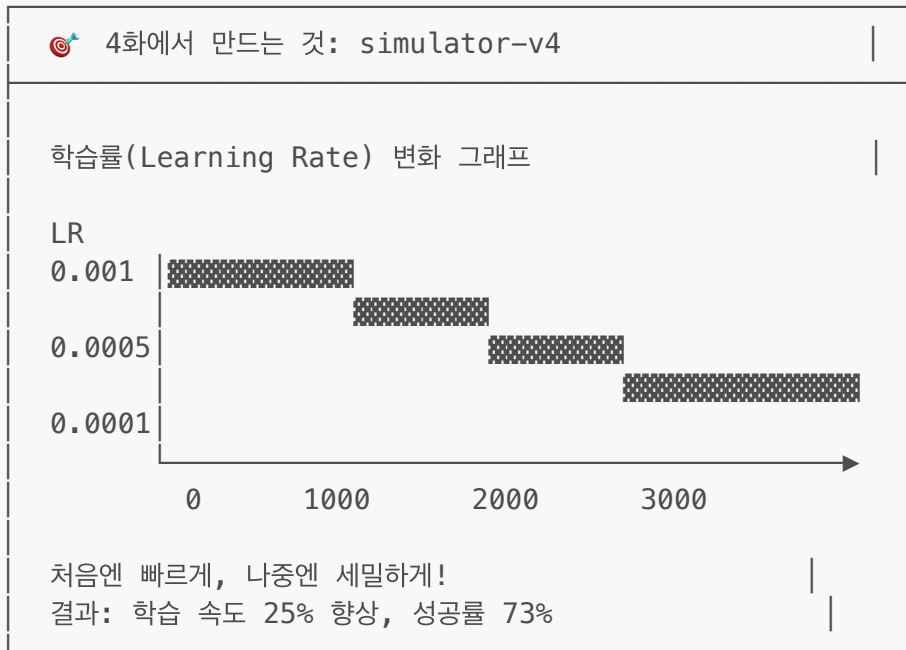
팀장님이 덧붙였습니다.

"그런데 학습이 3000번이나 걸리잖아요. 꽤 오래 걸리던데... 더 빨리 배우게 할 방법은 없을까요?"

메타코딩의 머릿속에 2화에서 배운 경사하강법이 스쳤습니다. "처음엔 큰 보폭, 나중엔 작은 보폭"이라는 전략.

"찾아보겠습니다!"

4화: 더 빠르게, 더 똑똑하게 (v4)



17 4주차 — 경사하강법의 실전 적용

v3를 완성하고 나서 메타코딩은 유튜브에서 강화학습 강의를 보다가 흥미로운 기법을 발견했습니다.

"Learning Rate Scheduling"

2화에서 경사하강법을 배울 때, "처음엔 큰 보폭으로 전체를 훑고, 나중엔 작은 보폭으로 정밀하게 찾는다"가 가장 효율적이라고 했죠. 그 전략을 실제 코드로 구현하는 것입니다!

🧠 Learning Rate Scheduling: 보폭 자동 조절

"처음엔 성큼성큼, 나중엔 살금살금!"

[비유: 새 교과서를 공부하는 방법]

1단계: 빠르게 훑어보기 (큰 Learning Rate = 0.001)

"우선 목차랑 소제목만 쪽 읽어보자!"

→ 전체 구조 파악!

→ "아, 이 책은 대충 이런 내용이구나!"

2단계: 본문 차근차근 읽기 (중간 Learning Rate = 0.0005)

"이제 각 챕터를 하나씩 읽어보자"

→ 핵심 개념 이해!

3단계: 중요 부분 정독 (작은 Learning Rate = 0.0001)

"이 부분은 핵심이니깐 세 번 읽자!"

→ 디테일까지 완벽히!

→ 처음부터 한 글자씩 정독하는 것보다 훨씬 효율적!

경사하강법의 산 비유로 돌아가면:

[학습률이 변하는 과정 - 산에서 최저점 찾기]

처음 (LR = 0.001, 큰 보폭):

손실 | ●
| ↘ ↘ ↘ ← 빠르게 대략적인 위치 찾기!
| ↓
| ● "이 근처가 가장 낮은 것 같아!"
| →

나중 (LR = 0.0001, 작은 보폭):

손실 | ●
| ↓ ↓ ↓ ← 조심조심 정밀하게!
| × "여기가 진짜 최저점!"
| →

🔧 v4 코드 소개

```
class DQNAgent:
    def __init__(self):
        self.initial_lr = 0.001    # 처음: 큰 보폭으로 빠르게
        self.min_lr = 0.0001       # 마지막: 작은 보폭으로 세밀하게

    def update_learning_rate(self, episode):
        # 에피소드가 진행될수록 학습률이 줄어듦
        decay_rate = 0.998
        self.current_lr = self.initial_lr * (decay_rate ** episode)
        self.current_lr = max(self.current_lr, self.min_lr)

        # 옵티마이저에 적용
        for param_group in self.optimizer.param_groups:
            param_group['lr'] = self.current_lr
```

`decay_rate = 0.998`은 매 에피소드마다 학습률을 0.2%씩 줄인다는 뜻입니다. 아주 조금씩, 천천히 보폭을 줄여나가는 것이죠.

🎮 v4 실행 결과

```
cd simulator-v4
python train.py
```

Episode 100		Score: 1.2		LR: 0.000820	← 아직 큰 보폭
Episode 500		Score: 3.8		LR: 0.000368	← 점점 줄어듦
Episode 1000		Score: 5.2		LR: 0.000135	← 중간 보폭
Episode 2000		Score: 6.1		LR: 0.000100	← 세밀한 보폭
Episode 3000		Score: 6.8		LR: 0.000100	← 최소값 유지



🧠 학습과 추론은 다르다

메타코딩은 다시 김선배를 찾아갔습니다.

"선배님, 팀장님이 추론 속도를 최적화하라고 하시는데... 추론이 뭔가요?"

김선배가 웃으며 화이트보드에 적었습니다.

AI의 두 단계:

학습 (Training):

- AI가 "배우는" 과정
- 공장에서 한 번 (몇 시간~며칠)
- 끝나면 모델 파일(.pth)로 저장
- 느려도 괜찮음!

추론 (Inference):

- 배운 걸 "실제로 사용하는" 과정
- 매번 청소할 때마다!
- 빠를수록 좋음! (배터리 절약, 반응 빠름)

사람으로 비유하면:

학습 = 운전면허 학원 다니기 (한 번)

- 몇 주 걸려도 괜찮음
- 열심히 배우면 됨

추론 = 매일 아침 출근 운전 (매일)

- 빠르고 자연스러워야 함
- 매번 교과서 펴보면서 운전할 수 없잖아요!

"학습은 한 번이면 되지만, 추론은 청소할 때마다 해야 하잖아요. 매번 신경망 45ms를 기다리는 건 비효율적이에요."

💡 메타코딩의 아이디어: 캐싱!

"그리고 보니... 같은 집에서 매일 청소하면 비슷한 경로로 다니는데, 매번 신경망 계산을 하는 게 낭비 아닌가? 이미 성공한 경로를 저장해두면 더 빠르지 않을까?"

네비게이션에 비유하면:

❌ 매번 새로 계산:
"서울→부산 경로를 다시 계산합니다... 45ms"
(매일 같은 길인데 매번 계산?)

✅ 자주 가는 길은 저장:
"저장된 경로입니다! 바로 안내합니다. 12ms"
(3.75배 빠름!)

🔧 캐싱 시스템 코드 소개

```
class ActionCache:
    """성공한 경험을 저장하는 캐시"""
    def __init__(self):
        self.cache = {} # 상태 → 행동 매핑

    def get(self, state):
        """이 상황을 본 적 있나?"""
        state_key = tuple(round(x, 2) for x in state)
        return self.cache.get(state_key, {}).get('action', None)

    def store(self, state, action, success):
        """성공한 행동만 저장!"""
        if success:
            state_key = tuple(round(x, 2) for x in state)
            self.cache[state_key] = {'action': action}
```

핵심은 **성공한 경험만 저장**한다는 것입니다. 실패한 경험까지 저장하면 "여기서 벽에 부딪혔었지"라는 나쁜 기억을 따라갈 수도 있으니까요.

💡 두 가지 전략: 정적 vs 동적 환경

김선배가 중요한 포인트를 짚어줬습니다.

"캐시를 쓸 때는 환경에 따라 전략을 달리해야 해요."

전략 1: 정적 환경 (공장, 창고 — 장애물이 고정)

캐시에 있으면 → 캐시 사용! (12ms, 3.75배 빠름!)
캐시에 없으면 → 신경망 계산 (45ms)

매일 같은 구조인 창고에서는 캐시가 매우 효과적!

전략 2: 동적 환경 (가정집 — 가구 이동, 사람 이동)

항상 신경망(Policy)이 먼저! (45ms, 하지만 안전!)
캐시는 참고만 → 캐시와 다르면 신경망 우선!

사람이 돌아다니는 집에서는 안전이 최우선!

운전자(Policy)가 네비(Cache)보다 우선인 이유:

네비: "이 길로 직진하세요"

운전자: "어? 앞에 공사중이야! 우회해야겠어!"

→ 실시간 판단(Policy)이 저장된 기억(Cache)보다 중요!

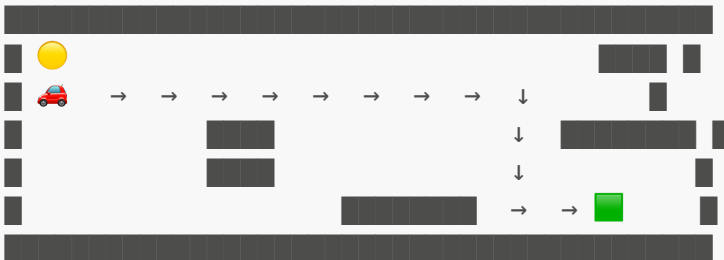
→ 눈앞의 현실이 과거의 기억보다 정확!

🎮 v5 실행 결과

```
cd simulator-v5
python main.py
```

정적 환경 (장애물 고정):

[v5 - 정적 환경]



Cache: Size=350 | Hits=142 | Agreements=89 | Conflicts=8

✅ Success Rate: 73%

캐시 덕분에 추론 속도 3.75배 향상!

동적 환경 (50스텝 후 새 장애물 등장!):

[v5 - 동적 환경, Step 51]





△ 캐시: "직진" vs Policy: "장애물 감지! 우회!"
→ Policy 우선으로 안전하게 우회! ✓

Success Rate: 65% (캐시만 믿었다면: 30%)

📦 최종 보고

팀장님: "좋아요! 정적 환경에서는 빠르고, 동적 환경에서도 안전하네요. 이제 제품화 고민을 시작할 수 있겠어요!"

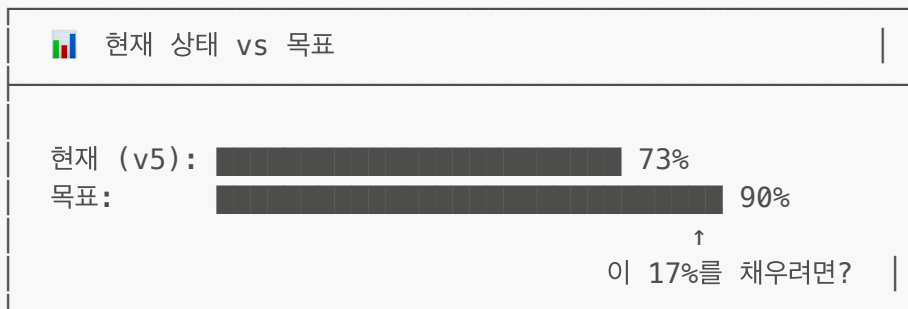
5주간의 여정... 드디어 제품화 가능 수준에 도달했습니다.

최종 성능 정리:

새로운 맵 성공률	: 73%
학습 속도 개선	: v3 대비 25% 향상 (v4)
추론 속도 개선	: 정적 환경에서 3.75배 빠름 (v5)

하지만 73%는 아직 부족합니다. 팀장님의 목표는 **90%** 이니까요.

6화: 73%에서 90%로 — 남은 여정



🤔 왜 73%에서 막혔을까?

v5까지 만들면서 많이 발전했지만, 아직 부족한 것들이 있습니다. 메타코딩은 김선배와 함께 문제를 하나씩 분석했습니다.

🔍 부족한 부분 1: 중요한 경험을 충분히 복습하지 않음

2화에서 경험 리플레이를 배웠죠? 현재는 모든 경험을 **동등하게** 뽑아서 학습합니다.

경험 참고 (현재):

경험 1: 이동 중 (평범)

경험 2: 이동 중 (평범)	← 이걸 더 자주 학습해야 하는데 랜덤이라 잘 안 뽑힘!
경험 3: 목적지 도달!! (귀하고 중요!)	
경험 4: 이동 중 (평범)	
경험 5: 이동 중 (평범)	

해결책: Prioritized Experience Replay (PER)

중요도 기반으로 뽑기!

경험 3: 목적지 도달!! (우선순위: ★★★)	← 자주 뽑힘! ← 가끔 뽑힘 ← 드물게 뽑힘
경험 7: 벽 충돌 (우선순위: ★★)	
경험 1: 이동 중 (우선순위: ★)	

핵심: "놀라운 경험"을 더 자주 복습!
AI가 예상을 많이 틀린 경험 = 배울 게 많은 경험

기대 효과: +5~8% 성공률 향상!

🔍 부족한 부분 2: 같은 신경망이 판단도 하고 평가도 함

현재 DQN 구조:

하나의 신경망 (Policy Network)

역할 1: "어떤 행동이 좋을까?" (판단)

역할 2: "그 행동이 얼마나 좋은가?" (평가)

문제:

자기가 좋다고 한 행동을 자기가 평가?

→ 자기 자신에게 후한 점수를 줄 수밖에 없음!

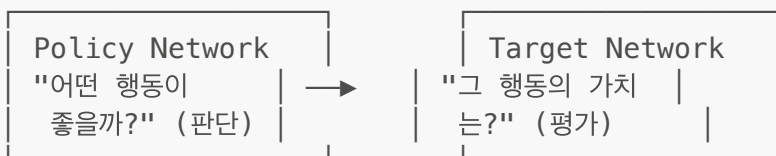
학교에 비유하면:

학생이 시험 문제를 내고, 자기가 채점하는 것!

→ 당연히 점수가 높게 나오겠죠?

해결책: Double DQN

개선된 구조:



선수가 플레이하고, 심판이 평가!
→ 더 객관적이고 정확한 학습!

기대 효과: +3~5% 성공률 향상!

🔍 부족한 부분 3: 신경망 구조가 단순함

현재 신경망:
입력(11개) → 은닉층(64개) → Q값(3개)

문제: 하나의 은닉층으로 모든 걸 처리하려니 버거움

해결책: Dueling DQN

Dueling DQN 구조:

입력(11개) → 은닉층 → $\begin{cases} \text{[상태 가치 V]} \\ \text{[행동 이점 A]} \end{cases} \rightarrow \text{Q값(3개)}$

V(상태 가치): "이 위치 자체가 얼마나 좋은가?"
예: 목적지 바로 옆 = 매우 좋은 위치!

A(행동 이점): "직진이 우회전보다 얼마나 나은가?"
예: 직진 +5, 우회전 -2, 좌회전 -3

$Q = V + A$
= "위치가 좋고" + "이 행동도 좋으면" = 높은 Q값!

비유: 부동산
V = "강남이 좋은 동네인가?" (위치)
A = "이 집이 같은 동네 다른 집보다 나은가?" (행동)
→ 위치와 선택을 분리해서 판단!

기대 효과: +4~7% 성공률 향상!

🔍 부족한 부분 4: 학습 맵이 20개뿐

현재: 20개 맵으로 학습
문제: 비슷한 패턴만 보다 보니 다양성 부족

해결: 100개 이상의 다양한 맵!

추가할 맵 유형:

미로형: 여러 갈림길 중 하나가 정답	
U자형: 반대 방향으로 먼저 가야 함	

나선형: 빙글빙글 돌아가야 함

열린형: 장애물 거의 없음

밀집형: 장애물이 매우 많음

기대 효과: +3~5% 성공률 향상!

🔍 부족한 부분 5: AI가 받는 정보가 적음

현재 v5 입력 (11개):

주변 8칸 + 방향 + 목적지 방향

문제: 바로 옆 한 칸만 봄

→ "두 칸 앞에 막다른 길"을 미리 못 봄!

해결: 시야 확장!

개선된 입력 (예시):

주변 1칸 (8개) ← 현재

주변 2칸 (16개) ← 확장!

현재 방향 (1개)

목적지 방향 (2개)

최근 이동 이력 (3개) ← 새로 추가!

기대 효과: +2~3% 성공률 향상!

📊 개선 로드맵

현재 v5: 73%

- ① Double DQN 적용: +3~5% → 76~78%
- ② Prioritized Replay 적용: +5~8% → 81~86%
- ③ Dueling Network 적용: +4~7% → 85~93%
- ④ 맵 다양성 증가 (100개+): +2~4% → 87~95%
- ⑤ 입력 정보 확장: +2~3% → 89~98%

목표: 90% 이상 달성 가능!

메타코딩은 노트를 덮으며 중얼거렸습니다.

"73%에서 90%까지... 아직 할 일이 많지만, 어디를 어떻게 고쳐야 하는지 이제 보인다."

그것이 가장 중요한 변화였습니다.

에필로그: 메타코딩이 진짜 배운 것

5주간의 여정을 돌아보며

금요일 저녁, 텅 빈 사무실. 메타코딩은 책상 위에 놓인 로봇 청소기를 바라보며 노트를 정리했습니다.

5주 전, 이 로봇 청소기를 처음 봤을 때의 막막함이 떠올랐습니다. 강화학습이 뭔지도 몰랐고, DQN은 이름만 들어봤고, 경사 하강법은 수업 때 좋아서 기억도 안 났습니다.

그런데 막상 해보니깐요.

전체 버전 비교표

버전	방식	성공률	새 맵?	핵심 교훈
v0	if 규칙	~50%*	❌	규칙만론 한계 (국소 최적화)
v1	DQN 기본	80%**	❌	첫 AI! 하지만 과적합
v2	+ 다양성	70%	✅	다양한 경험이 진짜 실력
v3	+ 공정 평가	68%	✅	훈련셋/테스트셋 분리
v4	+ LR 스케줄	73%	✅	처음엔 크게, 나중엔 작게
v5	+ 캐싱	73%	✅	학습 vs 추론 최적화
목표	고급 기법	90%+	✅	끊임없는 개선!

*v0: 단순한 맵에서만 성공 / **v1: 학습한 맵에서만 성공

이 책에서 배운 AI 개념 총정리

[기초 개념]

- ✅ 강화학습 (Reinforcement Learning)
 - 보상과 패널티로 스스로 배우는 AI
- ✅ DQN (Deep Q-Network)
 - 신경망으로 Q값을 계산하는 알고리즘
- ✅ Q값 (Q-value)
 - "이 상황에서 이 행동은 미래까지 얼마나 좋은가?"
- ✅ 상태 벡터 (State Vector)
 - 현재 상황을 숫자 묶음으로 표현한 것 (11개)
- ✅ 보상 함수 (Reward Function)
 - AI에게 "잘했어/안돼!"를 알려주는 선생님

[신경망 관련]

- ✓ **신경망 (Neural Network)**
 - 입력 → 은닉층 → 출력 (뇌를 수학으로 흉내)
- ✓ **가중치 (Weight)와 편향 (Bias)**
 - 뉴런의 연결 강도. AI가 학습하는 것의 정체!
- ✓ **활성화 함수 - ReLU**
 - 음수는 차단(0), 양수는 통과. 빠르고 효과적!
- ✓ **활성화 함수 - Sigmoid**
 - 0~1 사이로 압축. 확률 표현에 유리!

[학습 원리]

- ✓ **경사하강법 (Gradient Descent)**
 - 오차를 줄이는 방향으로 가중치를 조금씩 수정
- ✓ **기울기 / 그래디언트 (Gradient)**
 - 미분의 결과. "어느 방향으로 수정하면 오차가 줄까?"
- ✓ **역전파 (Backpropagation)**
 - 오차를 뒤에서 앞으로 전파하여 각 가중치 수정
- ✓ **손실 함수 (Loss Function)**
 - "AI가 얼마나 틀렸나?"를 수치화한 것
- ✓ **로컬 미니마 (Local Minima)**
 - 가짜 최저점. 더 낮은 곳이 있는데 여기서 멈춤
- ✓ **Learning Rate (학습률)**
 - 한 걸음의 크기. 크면 빠르지만 불안정, 작으면 안정적이지만 느림
- ✓ **Learning Rate Scheduling**
 - 처음엔 크게, 나중엔 작게. 효율적 학습!
- ✓ **Gradient Clipping**
 - 한 번에 너무 크게 변하지 않도록 기울기를 제한

[데이터와 평가]

- ✓ **과적합 (Overfitting)**
 - 연습 문제만 잘 풀고, 새 문제는 못 푸는 현상
- ✓ **일반화 (Generalization)**
 - 처음 보는 상황에서도 잘 작동하는 진짜 실력
- ✓ **훈련셋 / 테스트셋**
 - 공부용 문제와 시험 문제는 반드시 분리!

[탐험과 전략]

- ✓ **Epsilon-Greedy (탐험 vs 활용)**
 - 처음엔 탐험 많이, 나중엔 배운 것 활용
- ✓ **경험 리플레이 (Experience Replay)**
 - 과거 경험을 저장하고 랜덤으로 복습

- ✓ 미니배치 (Mini-batch)
 - 한 번에 32개씩 뽑아서 학습 (안정성 + 속도)
- [추론 최적화]
- ✓ 학습 vs 추론
 - 학습은 한 번(오래 걸려도 됨), 추론은 매번(빨라야 함)

- ✓ 캐싱 (Caching)
 - 성공 경험 저장으로 추론 속도 향상

- ✓ Policy > Cache 원칙
 - 실시간 판단이 저장된 기억보다 우선!

[고급 기법 (미리보기)]

- ✓ Double DQN → 판단과 평가를 분리
- ✓ Dueling DQN → 위치 가치와 행동 이점을 분리
- ✓ Prioritized Experience Replay → 중요한 경험을 자주 복습

💡 메타코딩이 진짜 배운 것

기술적인 것 말고, 이번 프로젝트에서 진짜 깨달은 것들이 있었습니다.

첫째: 실패가 교과서다

v1의 실패가 없었다면 과적합을 몰랐을 것입니다. v2의 착각이 없었다면 훈련셋/테스트셋 분리를 지나쳤을 것입니다. 모든 실패가 다음 버전을 만드는 재료가 됐습니다. AI도 실패에서 배우고, 개발자도 실패에서 배웁니다.

둘째: 한 번에 완벽할 필요 없다

v0 → v1 → v2 → v3 → v4 → v5 → ...

처음부터 v5를 만들려고 했다면?
→ 너무 복잡해서 포기했을 것

한 번에 하나씩 문제를 해결하면?
→ 결국 v5까지 도달!

이것은 AI의 학습 방식과도 똑같습니다. AI도 한 번에 완벽해지지 않습니다. 수천 번의 에피소드를 거치면서 조금씩, 조금씩 나아지죠.

셋째: "왜?"를 알면 "어떻게?"가 보인다

73%가 왜 나왔는지 이해했더니, 어떻게 90%로 올릴 수 있는지 자연스럽게 보였습니다. 문제의 원인을 아는 것이 해결의 시작입니다.

🚀 다음 책에서 배울 것들

이 책은 AI 세계의 **첫 번째** 문이었습니다. 문을 열었으니, 이제 더 넓은 세계가 기다립니다.

다음 단계 학습 로드맵:

- 📖 파이썬 심화
 - 이 책에서 본 코드들을 직접 짜보기!
- 📖 판다스 / 데이터 분석
 - 데이터를 정리하고 분석하는 기술
- 📖 수학 / 선형대수학
 - 신경망의 가중치 계산을 수학으로 완전히 이해
 - 행렬 곱셈이 신경망의 핵심!
- 📖 AI / 머신러닝
 - DQN 말고 다른 알고리즘들
 - 의사결정 트리 (Decision Tree): 질문으로 답을 찾아가는 AI
 - 랜덤 포레스트 (Random Forest): 여러 트리의 투표!
 - SVM, KNN 등 다양한 접근법
- 📖 신경망 / 딥러닝
 - CNN: 이미지를 이해하는 AI
 - RNN: 시간 순서를 이해하는 AI
 - Transformer: ChatGPT의 핵심 기술!
- 📖 LLM / RAG
 - ChatGPT 같은 대형 언어 모델의 원리
 - 문서 검색 + AI를 결합하는 기술

📖 마지막 페이지

메타코딩의 일기 마지막 장:

처음 팀장님이 청소 로봇 AI를 만들라고 했을 때, 저는 너무 무서웠어요.

강화학습이 뭔지도 몰랐고, DQN은 이름만 들어봤고, 경사하강법은 수업 때 졸아서 기억도 안 났고...

그런데 막상 해보니깐요.

한 번에 완벽하게 할 필요가 없더라고요. v0가 실패해도 괜찮았어요. 그 실패가 v1을 만들게 해줬으니까요. v1이 실패해도 괜찮았어요. 그 실패가 v2를 만들게 해줬으니까요.

AI가 처음인 여러분에게 말하고 싶어요. 실패를 두려워하지 마세요. 실패는 다음 버전의 씨앗입니다.

모르는 개념이 나와도 괜찮아요. "나중에 이해하면 되지" 하고 계속 읽어나가다 보면 어느 순간 전체 그림이 보이기 시작해요.

한 화씩, 한 버전씩. 여러분도 할 수 있습니다.

— 메타코딩

용어 사전

용어	한 줄 설명	쉬운 비유
강화학습	보상으로 배우는 AI	칭찬받으며 자라는 강아지
DQN	Q값 계산 신경망	AI의 뇌
Q값	행동의 예상 미래 점수	"이거 하면 얼마나 좋지?"
상태 벡터	현재 상황의 숫자 표현	AI의 눈 (11개 숫자)
보상 함수	행동의 점수를 매기는 규칙	AI의 선생님
신경망	뉴런들의 연결 구조	사람의 뇌를 흉내냄
가중치	뉴런 연결의 강도	시냅스의 세기
편향	뉴런의 기본 반응값	기본 성향
ReLU	음수→0, 양수→그대로	나쁜 신호 차단 필터
Sigmoid	0~1 사이로 압축	확률 변환기
경사하강법	오차를 줄이며 학습	안개 속 산 내려오기
Gradient	가중치 수정 방향	나침반
역전파	오차를 뒤로 전파	실패 원인 거꾸로 추적
손실 함수	오차 크기 측정	시험 점수 (낮을수록 좋음)
로컬 미니마	가짜 최저점	산속 작은 웅덩이
과적합	암기만 하고 진짜 실력 없음	시험문제 외워서 통과
일반화	새 상황에도 잘 작동	진짜 실력
Epsilon	탐험 확률	새 식당 vs 단골집
Learning Rate	학습 보폭 크기	한 걸음의 크기
LR Scheduling	보폭 자동 조절	처음엔 크게, 나중엔 작게
Gradient Clipping	기울기 크기 제한	급커브에서 핸들 제한
경험 리플레이	과거 경험 저장·복습	오답 노트
미니배치	한 번에 32개씩 학습	묶음 복습
훈련셋	학습용 데이터	연습 문제집
테스트셋	평가용 데이터	시험지
학습 vs 추론	배우기 vs 써먹기	면허 학원 vs 매일 운전
캐싱	성공 경험 저장으로 빠르게	네비 즐겨찾기
Policy	AI의 행동 규칙	운전자의 판단
Double DQN	판단+평가 분리	선수와 심판 분리

용어	한 줄 설명	쉬운 비유
Dueling DQN	위치+행동 분리 평가	부동산: 동네 vs 이 집
PER	중요 경험 우선 복습	오답 중 자주 틀리는 것 먼저

✅ 이해도 체크리스트

기본 개념

- ☐ if 규칙의 한계(국소 최적화)를 설명할 수 있다
- ☐ 강화학습이 왜 필요한지 알겠다
- ☐ Q값이 무엇인지 한 문장으로 말할 수 있다
- ☐ 신경망의 입력-은닉-출력 구조를 알겠다
- ☐ 뉴런 하나가 어떻게 계산하는지 이해했다

활성화 함수와 학습

- ☐ ReLU가 하는 일을 알겠다
- ☐ Sigmoid가 하는 일을 알겠다
- ☐ 경사하강법을 산 비유로 설명할 수 있다
- ☐ Gradient(기울기)가 미분이라는 것을 이해했다
- ☐ 역전파가 왜 필요한지 알겠다
- ☐ 로컬 미니마가 뭔지 이해했다

데이터와 학습 전략

- ☐ 훈련셋과 테스트셋을 왜 분리하는지 알겠다
- ☐ 과적합이 왜 위험한지 이해했다
- ☐ Exploration vs Exploitation을 이해했다
- ☐ 경험 리플레이가 왜 필요한지 알겠다
- ☐ Learning Rate가 학습에 미치는 영향을 안다

실무 개념

- ☐ Learning Rate Scheduling의 원리를 안다
- ☐ 학습과 추론의 차이를 알겠다
- ☐ Policy > Cache 원칙을 이해했다
- ☐ 73%에서 90%로 올리는 5가지 방법을 알겠다

모두 체크했나요? 축하합니다! 이제 여러분은 AI의 기초를 이해한 사람입니다!

끝.

"메타코딩과 함께한 5주간의 여정, 정말 수고하셨습니다!"